

Правила организации интерфейсных модулей

Данный набор правил позволит вам правильно спроектировать интерфейсный модуль, который будет готов к платформенным алгоритмам паковки и раздельного обновления.

Используйте папки для объединения JS-модулей в функциональные области

Функциональная область – это набор инструментов, которые используются для решения общей задачи. Типовые примеры таких областей — реестры и диалоги редактирования записей:

- MyModule/Registry/Browser
- MyModule/Document/Dialog

Пример из Платформы — пространства имен **Types/collection**:

- Types/collection:Enum
- Types/collection:Flags
- Types/collection:List
- Types/collection:RecordSet

Группируйте переиспользуемый код

Группируйте переиспользуемый между разными функциональными областями код в отдельных папках. Например, вы можете создать набор вспомогательных утилит и разместить их в папке Util:

- MyModule/Util/Constants
- MyModule/Util/Handlers

Интерфейсный модуль как коллекция библиотек

Рассматривайте интерфейсный модуль как коллекцию библиотек, каждая из которых экспортирует публичные модули:

- пример:

```
1 // MyModule/_foo/bar.ts модуль
2 export default function() {
3     return function Bar() {};
4 }
5
6 //MyModule/_foo/baz.ts модуль
7 export default class Baz {
8     constructor() {}
9 }
```

Библиотеки (а не модули!) должны быть импортированы вашими потребителями:

- пример:

```
1 // MyModule/foo.ts библиотека
2 export { default as Bar } from './_foo/bar';
3 export { default as Baz } from './_foo/baz';
```

[Подробнее о работе с TypeScript библиотеками](#)

Не используйте лишние папки

Практика показывает, что папка *components* не нужна, т.к. интерфейсный модуль, как правило содержит, только компоненты. "Не компоненты" можно разместить в отдельные папки или подпапку функциональной области.

Не используйте повторы слов в пути

- Плохо:
 - MyModule/Registry/Registry
 - MyModule/Document/Controller/AbstractController
- Хорошо:
 - MyModule/Registry/Embed
 - MyModule/Controller/Abstract

Работа с TypeScript библиотеками

Библиотека — это набор приватных и публичных модулей, которые сгруппированы в один файл по определенному критерию общности.

Использование TypeScript библиотек имеет ряд преимуществ:

- Нет необходимости использовать `bundles.js`;
- Уменьшается стек вызовов `RequireJS`;
- Автоматическая паковка в `production`-режиме.

Чтобы модули работали как библиотеки, они должны быть написаны на TypeScript.

Терминология

Приватная часть библиотеки - набор модулей, которые инкапсулирует библиотека. Эти модули недоступны снаружи.

Публичная часть библиотеки - набор модулей, которые экспортирует библиотека. Это публичное API.

Приватный модуль библиотеки - модуль, имя файла или название любой из папок, в которых он находится. Для такого модуля имена файла и папки должны начинаться с символа `_`.

Правила работы с библиотеками

1. Приватные модули можно подключить только в рамках библиотеки, в которую они входят. Подключение таких модулей вне библиотеки недоступно.
2. Подключите библиотеку целиком и извлеките из её публичной части нужный модуль.
3. Контроль и шаблон принадлежат одной библиотеке. Подключение контроля в шаблон выполняют как приватный модуль с полным путем.
4. Если у вас есть библиотеки внутри библиотек, следите, чтобы у них не было общих импортируемых модулей. В ином случае такие модули запакуются во все библиотеки. Вместо этого более крупные библиотеки должны импортировать модули из более мелких.

Рекомендации по работе с библиотеками

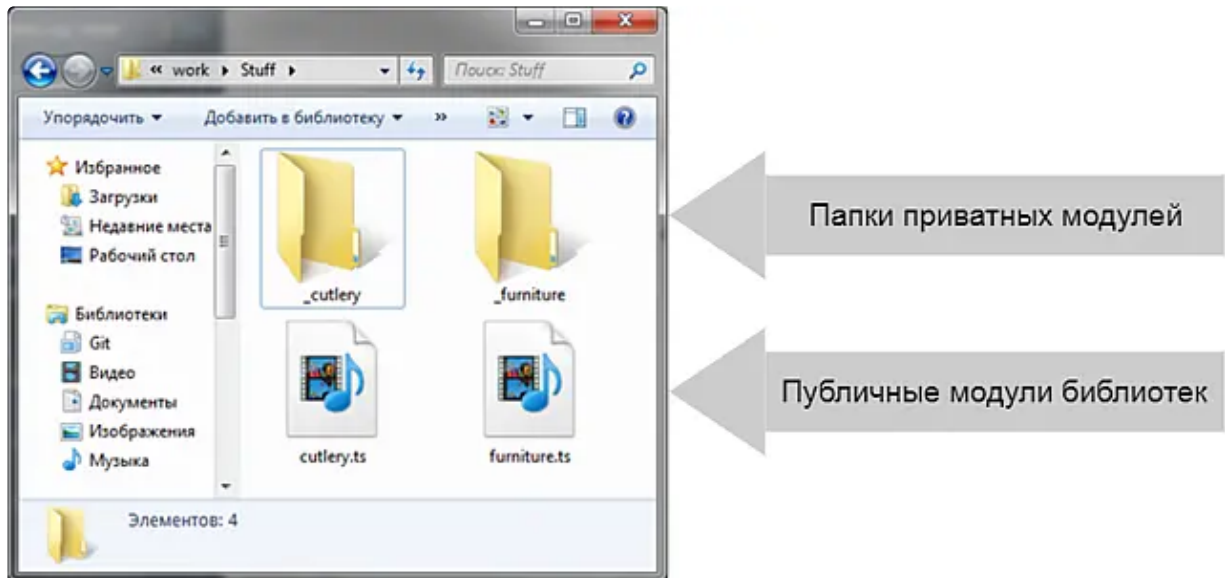
1. Все приватные модули библиотеки следует складывать в одноименную папку, начинающуюся с символа `"_"`.
2. Сами библиотеки должны называться с маленькой буквы, т.к. по сути это синглтоны (объекты, которые экспортируют модули).

Организация CSS-пакетов в рамках библиотек

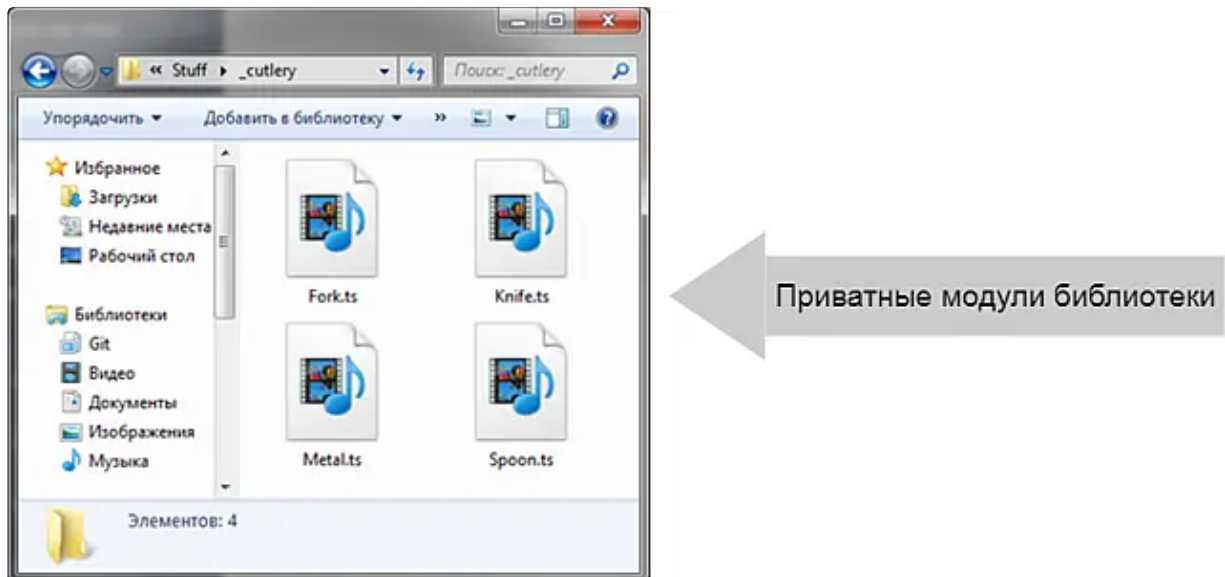
См. статью [CSS-пакеты для библиотек](#)

Пример организации библиотеки

Есть интерфейсный модуль **Stuff**, внутри которого лежат две библиотеки — **cutlery** и **furniture**.



Внутри папки **_cutlery** расположены частные модули библиотеки — **Fork.ts**, **Knife.ts**, **Metal.ts**, **Spoon.ts**.



Рассмотрим файл **cutlery.ts**.

```
1  /**
2   * Cutlery library
3   * @library Stuff/cutlery
4   * @includes Spoon Stuff/_cutlery/Spoon
5   * @includes Fork Stuff/_cutlery/Fork
6   * @includes Knife Stuff/_cutlery/Knife
7   */
8
9  // Публичная часть (экспортируемые модули).
10 export {default as Spoon} from './_cutlery/Spoon';
11 export {default as Fork} from './_cutlery/Fork';
12 export {default as Knife} from './_cutlery/Knife';
```

где:

- **@library** — доклет библиотеки;
- **@includes** — показывает, какие модули входят в библиотеку.

В данном файле происходит экспорт всех модулей, кроме **Metal**, поскольку он частный. Соответственно, мы можем выделить публичную часть (**Spoon**, **Fork**, **Knife**) и частную (**Metal**).

Рассмотрим частный модуль **Stuff/_cutlery/Fork.ts**. Нет принципиальных отличий по сравнению с обычным модулем.

```

1  /**
2   * Fork class
3   * @class Stuff/_cutlery/Fork    // Видно, что это приватный модуль.
4   * @extends Stuff/_cutlery/Metal
5   */
6
7  // Импорт приватного модуля этой же библиотеки.
8  import Metal from './Metal';
9
10 export default class Fork extends Metal /** @lends
    Stuff/_cutlery/Fork.prototype */ {
11   constructor() {
12     super();
13     //...
14   }
15 }

```

По имени ts-файла можно сделать вывод, что модуль Fork приватный, поскольку наименование папки начинается с символа "_".

Рассмотрим пример импорта публичного модуля из другой библиотеки:

- **Пример правильного подключения.** Достаточно подключить библиотеку целиком и "забрать" у нее нужный модуль.

```

1  /**
2   * Chair class
3   * @class Stuff/_furniture/Chair
4   * @extends Stuff/_furniture/Composite
5   */
6
7   import Composite from './Composite';
8   import {Knife} from '../cutlery'; // Импорт модуля Knife из
    библиотеки Stuff/cutlery.
9
10  export default class Chair extends Composite /** @lends
    Stuff/_furniture/Chair.prototype */ {
11    private _unexpectedKnife: Knife;
12
13    constructor() {
14      super();
15      this._unexpectedKnife = new Knife();
16      //...
17    }
18  }

```

- **Пример неправильного подключения.** Обратите внимание, что нельзя напрямую подключать модули из приватной папки чужого модуля и забирать оттуда нужный модуль. Подобные обращения к чужим библиотекам приведут к падению сборки.

```

1  /**
2   * Chair class
3   * @class Stuff/_furniture/Chair
4   * @extends Stuff/_furniture/Composite
5   */
6
7   import Composite from './Composite';
8   import Knife from '../_cutlery/Knife'; // Так делать нельзя.
9
10  export default class Chair extends Composite /** @lends

```

```

11     Stuff/_furniture/Chair.prototype */ {
12         private _unexpectedKnife: Knife;
13         constructor() {
14             super();
15             this._unexpectedKnife = new Knife();
16             //...
17         }
18     }

```

Примеры использования TypeScript-библиотек

Статическое подключение в ECMAScript-модуле

Импортируем модуль библиотеки.

```
import {Chair} from 'Stuff/furniture';
```

Динамическое подключение в ECMAScript-модуле

Подключение через инструкцию await:

```

1  import {IoC} from 'Env/Env';
2  ...
3  try {
4      const {Chair} = await import('Stuff/furniture');
5      // Do something with Chair
6  } catch (err) {
7      IoC.resolve('ILogger').error('My/Component', err);
8  }

```

Подключение через Promise:

```

1  import {IoC} from 'Env/Env';
2  ...
3  import('Stuff/furniture').then(({Chair}) => {
4      // Do something with Chair
5  }).catch ((err) => {
6      IoC.resolve('ILogger').error('My/Component', err);
7  });

```

Динамическое подключение по имени из библиотеки в ECMAScript-модуле

Для динамического подключения модуля по библиотечному имени используйте **Core/library:load**.

Подключение через инструкцию await:

```

1  import {IoC} from 'Env/Env';
2  import {load as loadModule} from 'Core/library';
3  ...
4  try {
5      const Chair = await loadModule('Stuff/furniture:Chair');
6      // Do something with Chair
7  } catch(err) {

```

```
8   IoC.resolve('ILogger').error('My/Component', err);
9 }
```

Подключение через Promise:

```
1 import {IoC} from 'Env/Env';
2 import {load as loadModule} from 'Core/library';
3 ...
4 loadModule('Stuff/furniture:Chair').then((Chair) => {
5     // Do something with Chair
6 }).catch((err) => {
7     IoC.resolve('ILogger').error('My/Component', err);
8 });
```

Подключение в AMD-модуле

Объявляем модуль, подключаем библиотеку целиком, достаем из нее нужный модуль.

```
1 define(
2     'My/Incredible/Module',
3     ['Stuff/furniture'],
4     function(furniture) {
5         var Chair = furniture.Chair;
6         ...
7     }
8 );
```

Подключение в шаблоне

Используем разделитель ":". По правилам XML двоеточие можно использовать при указании пространства имен только один раз, библиотека тоже всегда одна.

```
1 <Stuff.furniture:Chair material="leather"/>
2
3 <Controls.popupTemplate:CloseButton caption="Eat me"/>
```

Чтобы в шаблоне библиотеки подключить контрол из этой же библиотеки, необходимо подключать его как приватный модуль с полным путем, поскольку шаблоны конвертируются в AMD-модули.

Например, в шаблоне **MyPackage/myLibrary/_nsOne/myTemplate.wml** подключим модуль **MyPackage/myLibrary/_nsTwo/myComponent**:

```
1 <MyPackage.myLibrary._nsTwo.myComponent />
```

В случае использования синтаксиса подключения через имя тега, модуль **myComponent** является явной зависимостью шаблона **myTemplate**. Этот факт позволяет сборщику ресурсов при сборке библиотеки пробросить эту зависимость в шаблон из внутренней переменной библиотеки.



Нельзя указывать контрол как публичный модуль таким образом:

```
<MyPackage.myLibrary:nsTwo.myComponent />
```

Поскольку это приведет к циклической зависимости **MyPackage/myLibrary** → **wml!MyPackage/myLibrary/_nsOne/myTemplate** → **MyPackage/myLibrary**



Все остальные виды ссылок на ресурсы должны указывать на **публичные части библиотеки**. Например,

когда вы указываете обработчик, нельзя указывать приватную функцию:

```
1 <MyPackage.myLibrary._nsTwo.myComponent>
2   <ws:onDataLoad
3     type="function">MyPackage/myLibrary/_nsOne/onLoad</ws:onDataLoad>
4 </MyPackage.myLibrary._nsTwo.myComponent>
```

Вместо этого надо указать на экспортируемую библиотекой функцию:

```
1 <MyPackage.myLibrary._nsTwo.myComponent>
2   <ws:onDataLoad
3     type="function">MyPackage/myLibrary:nsOne.onLoad</ws:onDataLoad>
4 </MyPackage.myLibrary._nsTwo.myComponent>
```



Нельзя использовать конструкцию с атрибутом **data-component** и указанием на приватный модуль — такая зависимость тоже не является явной:

```
<div data-component="MyPackage/myLibrary/_nsTwo/myComponent" />
```

Ее надо брать из публичной части библиотеки:

```
<div data-component="MyPackage/myLibrary:nsTwo.myComponent" />
```

Подключение контрола-наследника Lib/Control/CompoundControl/CompoundControl из приватного модуля библиотеки

Отдельное решение требуется использовать для контролов, унаследованных от [Lib/Control/CompoundControl/CompoundControl](#).

Если вы вставляете подобный контрол как приватный модуль библиотеки, то:

- данный контрол необходимо экспортировать из библиотеки;
- требуется указать атрибут **amdModuleName** с публичным именем этого контрола.

Например, если **MyPackage/myLibrary/_nsTwo/myComponent** унаследован от **CompoundControl**, то его подключение будет выглядеть так:

```
<MyPackage.myLibrary._nsTwo.myComponent
  amdModuleName="MyPackage/myLibrary:nsTwo.myComponent" />
```

Подобное решение является компромиссом, обеспечивающим возможность создавать контролы на базе **CompoundControl**.